How WBR perception works

A wheel-biped robot balances on two wheels at the ends of articulated legs. It needs to understand its motion while its controller keeps it upright.

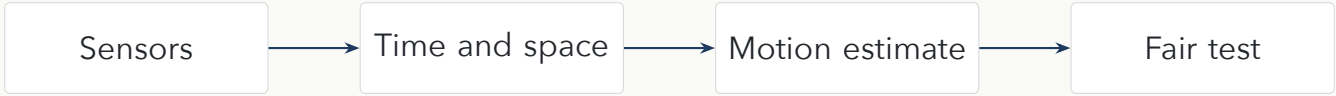


Conceptual illustration: follow one camera moment through the system.

From sampled signals to an estimate of motion, then to a fair test.

We will follow one measurement from sensor to score

Imagine the robot turning past a table. The camera takes a left and right image of the same corner. Its motion sensor takes samples between images.



First we learn what the signals mean. Then we learn how software combines them, how to judge the answer, and why the project tests each layer in order.

Reading map: sensors, pages 6–12; projector detail, 9–11; gate learning checklist, 44–45; appendix, 46–49.

The same table corner will return throughout the deck.

The robot needs both its own motion and nearby geometry

Wheel rotation alone can mislead us when a wheel slips. A camera can see the table corner move across its image even when wheel measurements are wrong.

A **pose** is a position and an orientation: where the sensor is, and which way it points. A sequence of poses is a **trajectory**, or path through time.

Depth tells us how far away a visible surface is. Together, motion and depth help the robot reason about obstacles and where it can move.

Seeing a table is useful only if we know where the camera was when it saw it.

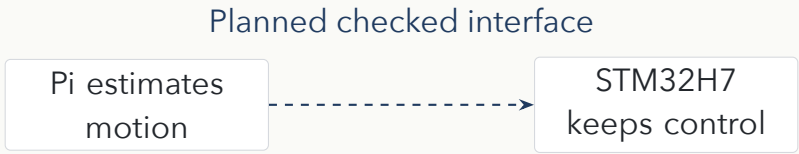
Balance must survive a delayed perception computer

The **STM32H7** owns motor safety and the hard real-time control loop. Hard real-time means a deadline must be met to preserve the required behavior.

Think of a timer interrupt that must update control before the next tick.

The **Raspberry Pi** runs perception on Linux. Image processing can take variable time, and the operating system can delay a task.

Its estimates must be checked for freshness and validity before a future controller interface uses them.



An old or missing pose must not become an unsafe motor command.

You can now explain why perception and control are separate

You should now be able to explain the following in your own words.

- A pose describes position and orientation; depth describes distance to a surface.
- The Pi can help understand the scene while the STM32H7 keeps responsibility for control and safety.
- Explain what could happen if a controller treated an old pose as a current measurement.

Try answering aloud before you turn the page.

The D435i combines two views with a motion sensor



Stylized D435i housing; the aperture drawing is not a hardware guide.

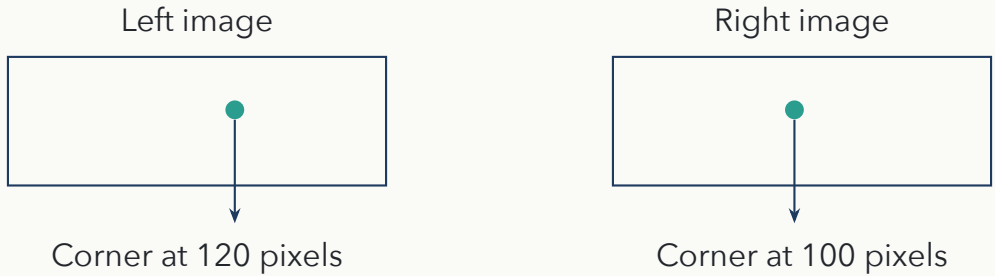
Two **infrared cameras** form a stereo pair: left and right views from different positions. The **IMU**, or inertial measurement unit, contains a gyroscope and an accelerometer.

In our running moment, both images see the table corner. The IMU measures the camera's turning and acceleration between image times.

The images and IMU provide different kinds of information about the same motion.

Stereo depth comes from a corner shifting between views

Disparity is the horizontal difference between the positions of the same point in the left and right images. Nearby points shift more than distant points.



Illustrative example: rectification aligns matching image rows. Focal length sets the projection scale in pixels. With $f = 400$ pixels and camera separation $b = 0.05$ metres, disparity $d = 20$ pixels gives depth $Z = fb/d = 1$ metre.

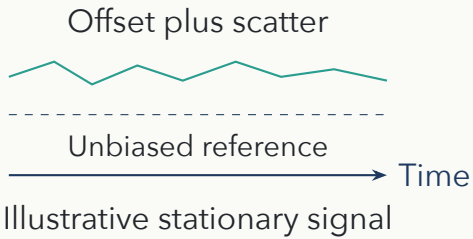
At 10 pixels of disparity, the same example gives 2 metres of depth.

IMU readings are useful signals with noise and bias

A **gyroscope** measures angular velocity: how fast the camera turns. An **accelerometer** measures specific force: acceleration relative to free fall. At rest on a table it reads about one g upward; in free fall it reads near zero. Here g is gravitational acceleration.

Noise is the rapid random variation, like ADC readings that scatter around a steady input.

Bias is an offset that may change slowly, like an ADC zero error.

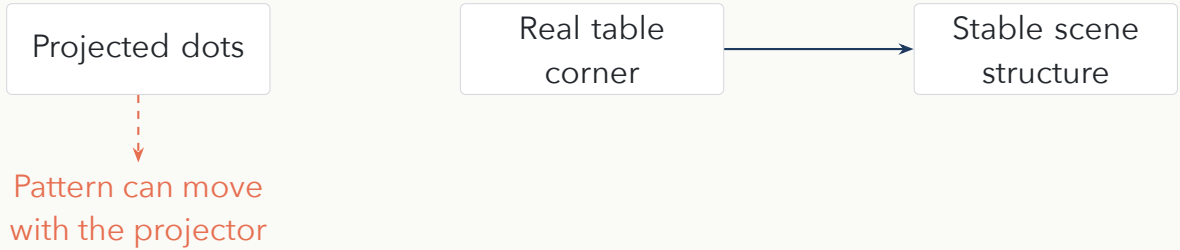


Illustrative stationary signal

A gyro bias adds a little false rotation at every time step, so the angle error grows.

The projector is off so features belong to the room

A **feature** is a recognizable image detail, such as a corner. The infrared projector adds dots that can help the depth system match points on a plain surface.



For visual motion estimation, we want persistent details attached to the room. Projected dots can change on surfaces as the camera moves and create misleading matches.

Turning off the projector trades artificial texture for more trustworthy natural texture.

The projector gives a plain wall texture for stereo depth

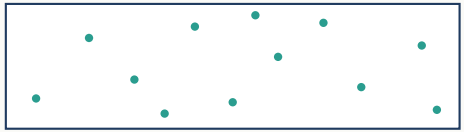
The D435i has a left infrared (IR) camera, a right IR camera and an IR projector. The cameras capture images; the projector produces no image. It shines an invisible dot pattern onto surfaces.

White wall with projector OFF



No distinctive detail to match.

White wall with projector ON



Both IR cameras see added detail.

Illustrative IR views: the dots are invisible to our eyes.

A plain wall can make left–right matching fail. Dots can make matching possible within the same stereo pair. For rectified images, disparity d is the horizontal pixel shift of a matched point.

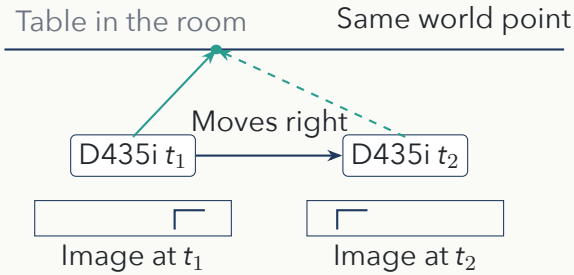
Depth is $Z = fb/d$: f is focal length in pixels; b is the distance between cameras (about 5 cm in our unit's nominal config); d is disparity in pixels. Z uses the same length unit as b.

Dots help identify a match; camera geometry turns its pixel shift into depth.

A projected dot can slide while a real corner stays fixed

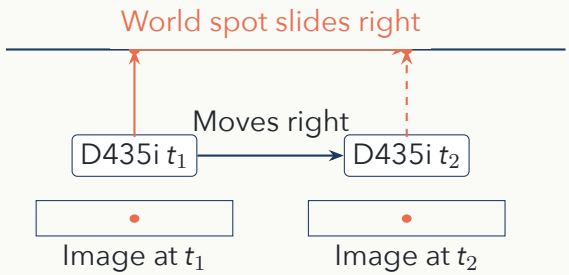
ORB-SLAM3 estimates motion from fixed room features. A sliding dot can give wrong motion. Here the camera moves right above a flat table without turning.

Physical table corner



Corner pixel shifts left.

Projected IR dot



Dot pixel stays here in this example.

Stereo depth on textureless surfaces	Often helpful.
ORB-SLAM3 tracking across time	Potentially misleading.
Our first natural-feature SLAM tests	OFF.

The attached projector pattern follows the camera; its lit spots slide in the room. OFF is experimental: later compare ON and OFF for tracking quality and depth.

If the D435i moves right, does the table corner move in the room, or only in the image?

You can now explain what each sensor contributes

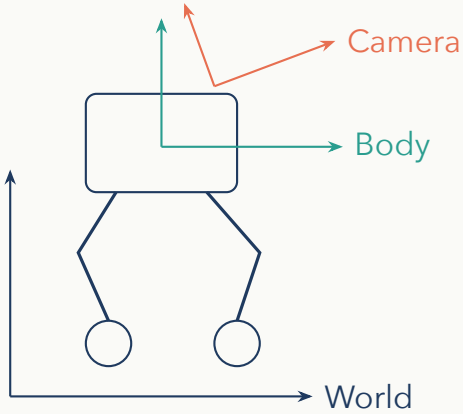
You should now be able to explain the following in your own words.

- Two views turn a pixel shift into depth when camera geometry is known.
- The gyro measures turning, and the accelerometer responds to motion and gravity.
- Explain why random noise, slowly changing bias and moving projected dots cause different problems.

Try answering aloud before you turn the page.

A coordinate frame says what a position is relative to

A **coordinate frame** is an origin and a set of axes. The same table corner has different coordinates relative to the camera, robot torso and room.



A **transform** converts coordinates between frames by rotating the axes and shifting the origin.

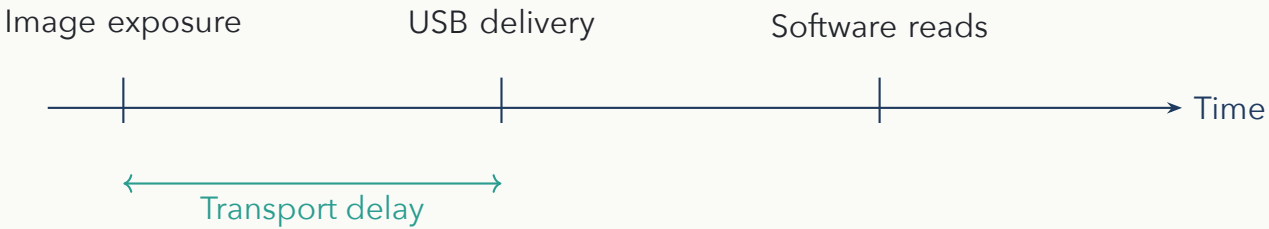
Illustrative example: if the camera is 0.2 m ahead of the torso and their axes agree, a corner 1.0 m ahead of the camera is 1.2 m ahead of the torso.

This schematic wheel-biped drawing shows two axes per frame to keep the geometry readable.

Never compare positions until they describe the same frame.

A timestamp names the measurement time, not its arrival

A **timestamp** is a clock reading attached to a measurement. Like capturing a timer value at an interrupt, it tells us when the event happened.

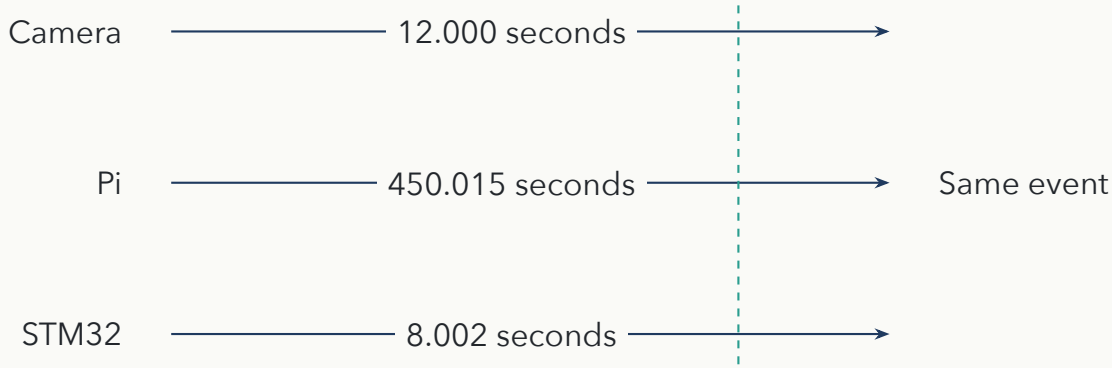


Our stereo pair belongs at its exposure time. Motion samples must cover the interval since the previous image, even if the Pi receives some of them later.

Using arrival time would turn scheduling delays into apparent sensor motion.

Different clocks need a measured relationship

A **clock domain** is the time reference used by a clock. The camera, Pi and STM32 can each start counting at a different origin and run at slightly different rates.



Illustrative example: for this event, Pi time equals camera time plus 438.015 s. This **offset** relates their origins. If their rates differ, the offset changes; one paired reading cannot measure that drift.

Record the clock domain as carefully as the timestamp value.

You can now explain where and when a sample belongs

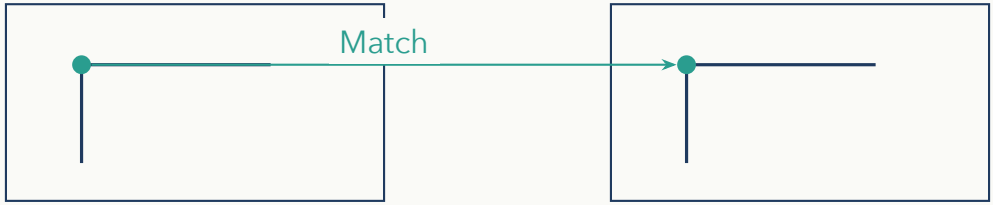
You should now be able to explain the following in your own words.

- A frame names the axes and origin; a transform connects two frames.
- A timestamp names a measurement event, while arrival time includes delivery delay.
- Explain why subtracting two readings from unrelated clocks does not give a time delay.

Try answering aloud before you turn the page.

Features let the computer recognize the same corner

A **feature** is a distinctive small image region, such as our table corner. ORB describes a region with bits from pixel-brightness comparisons. Think of a short binary signature for our corner.

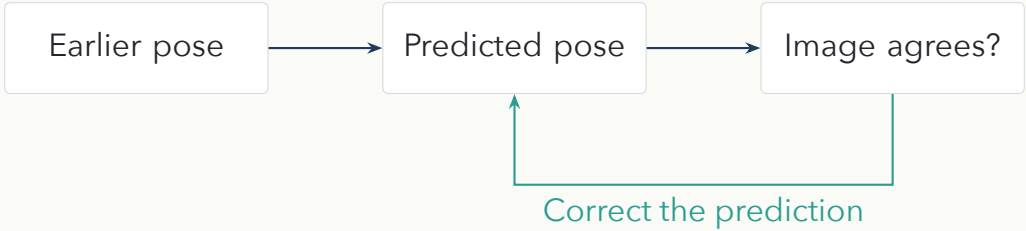


Feature matching means deciding which regions show the same physical point. Similar signatures suggest a match. Many matches must also agree with one camera motion before software trusts them.

The table corner becomes a repeatable observation rather than a whole picture to memorize.

Tracking estimates the pose that explains the image

Tracking means estimating the current pose from recent measurements. If the camera turns, the table corner appears at a new pixel position.



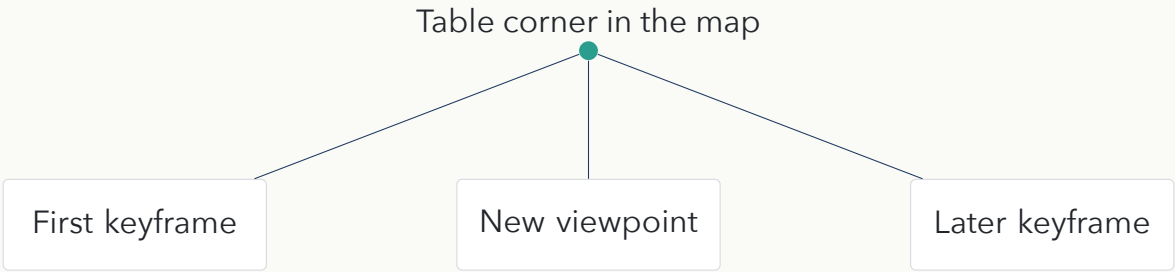
The estimator predicts where previously estimated scene points should appear, compares those predictions with observed pixels, and adjusts the pose to reduce the mismatch.

This resembles a control observer: predict the state, then use a measurement to correct it.

A pose is an estimate supported by many image matches, not a direct camera output.

Keyframes and map points preserve useful geometry

A **keyframe** is an image and its pose, kept because they add useful information. A **map point** is an estimated three-dimensional scene point observed in multiple images.



SLAM means simultaneous localization and mapping: estimating sensor poses and scene geometry together. Visual-inertial odometry estimates motion from images and IMU samples; SLAM also reuses a map and recognizes old places.

Stereo locates our corner in 3D; seeing it again helps locate the camera. Both estimates improve together.

The IMU predicts motion between camera images

Images arrive less often than IMU samples. **IMU preintegration** summarizes the turning and motion measured between two image times, instead of revisiting every sample for every pose correction.



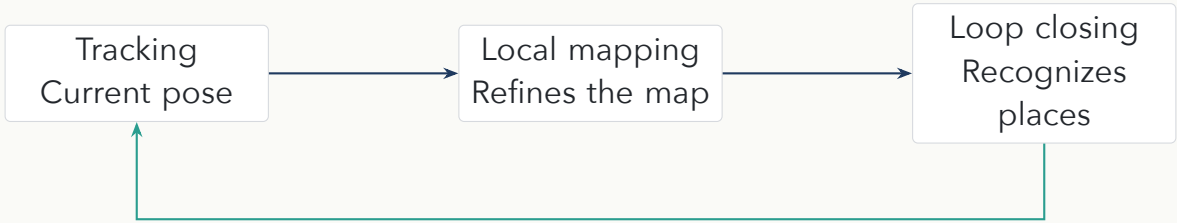
The IMU helps during rapid motion and helps estimate gravity. Stereo supplies distances in metres, rather than arbitrary units, through its known camera separation.

IMU initialization estimates starting velocity, gravity direction and sensor biases. Turning and accelerating in different directions help separate gravity from motion and sensor bias.

Images correct growing IMU motion error; this accumulated error is called drift.

Three threads share the work and recognize old places

A **thread** is a software task that can run alongside other tasks. Our SLAM software, **ORB-SLAM3**, separates urgent tracking from slower map improvement and place recognition.



Loop closing recognizes a place seen earlier. Returning to our table gives a new constraint: both visits should place the table at the same map location, so accumulated drift can be corrected.

Different responsibilities share one map; concurrent work does not guarantee a timing deadline.

Tracking fails when observations stop constraining motion

Calibration measures sensor properties so software can interpret the readings correctly.

- Blur smears corners, and a blank wall offers few distinctive features to match.
- Wrong calibration predicts the wrong pixels even for the correct pose.
- Wrong time pairing combines an image with motion from a different instant.
- A poorly initialized IMU can pull an otherwise plausible visual estimate in the wrong direction.

In our running moment, a missed corner match is different from pairing the corner with the wrong turn. The symptom may look similar, but the layer to inspect is different.

Check input quality, time and geometry before changing algorithm settings.

You can now explain how images and IMU produce a pose

You should now be able to explain the following in your own words.

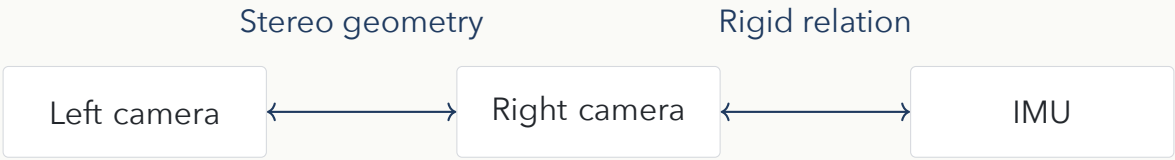
- Features identify observations; tracking finds a pose that explains them.
- Keyframes and map points preserve geometry, and loop closing constrains drift.
- Explain how preintegration and initialization make IMU samples useful, and why bad inputs can still defeat SLAM.

Try answering aloud before you turn the page.

Calibration tells software what the sensor geometry means

Intrinsics describe one camera's projection: focal length sets the pixel scale, the principal point is where the central viewing line of the camera meets the image, and distortion models lens warping.

Extrinsics describe the rotation and translation between sensors. Stereo needs the relation between cameras; visual-inertial estimation also needs the camera-to-IMU relation.



Check that the sensor parameters describe this camera and predict its observed pixels correctly.

Calibration must also describe time and uncertainty

A **camera–IMU time offset** describes how far the image and motion timelines are shifted relative to each other. Sharing a clock domain alone does not establish their physical alignment.

IMU noise parameters tell the estimator how much to trust samples and how quickly bias can change.



Conceptual target; not a calibration grid.

Kalibr fits camera and IMU geometry and timing from motion past a measured target. **Allan deviation** analyzes a long stationary IMU recording to estimate noise and slow bias behavior.

If IMU calibration is unavailable, assumed parameters still need measurement before we trust them.

You can now explain why calibration comes before trust

You should now be able to explain the following in your own words.

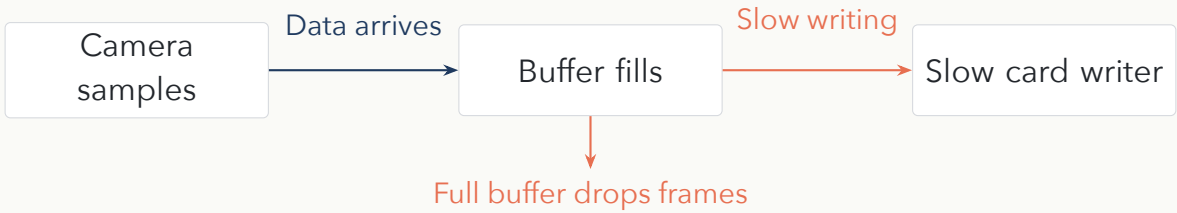
- Intrinsics connect a three-dimensional point to a pixel, and extrinsics connect sensor frames.
- Camera–IMU time offset and noise parameters describe different sources of error.
- Explain why a room-motion test and a measured-target calibration recording answer different questions.

Try answering aloud before you turn the page.

USB delivery and recording must keep up with the camera

The stereo pair and motion samples travel over local USB to the Pi. **librealsense** is the software development kit (**SDK**) that exposes images, motion readings and their metadata to programs.

A **buffer** temporarily holds data waiting to be written, like a UART receive queue. If the writer is slower than the arriving stream, the buffer fills and new frames are dropped.



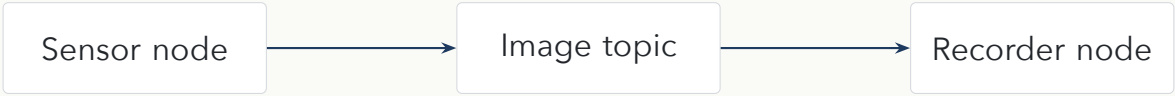
Our 100-second take had long gaps. The card averaged 16.6 MB/s, below the 18.4 MB/s needed. A writer that could not keep up is the likely cause.

We now record short takes into Pi RAM, then copy to the card and verify the copy. A 15-second RAM test had no dropped frames. RAM is temporary: power loss can erase the take before copying.

Record locally on the Pi; a remote shell controls the session.

ROS 2 messages make sensor data available to other tasks

ROS 2 is robotics middleware: it connects software components. A **node** is a participating program; a **topic** is a named channel carrying a defined message type.



The sensor node publishes each image; the recorder subscribes to receive it. The image message contains pixels and a header naming its time and coordinate frame. Other messages carry motion samples and calibration. Like a packet format, the metadata matters as much as the payload.

A topic name alone cannot prove that its messages have the right times or coordinate frames.

Recording once lets us replay the same problem

A **bag** is a file-based recording of streams and their timing information. Replaying it lets us test identical input repeatedly, like saving a logic-analyzer trace instead of reproducing an intermittent interrupt.

In the ROS 2 route, the driver publishes topics and rosbag2 records them. A separate input program must pass replayed images and IMU samples to SLAM.

In the direct route, librealsense records the stereo pair and IMU samples. The D435i input program reads that recording and passes them to ORB-SLAM3.



Conceptual review of recorded input.

Offline processing reads saved input and can wait for computation. **Live** processing reads arriving samples and must keep up. A recording for one route may not be readable by the other.

Record first to separate input problems from live scheduling problems.

You can now trace our moment through the software

You should now be able to explain the following in your own words.

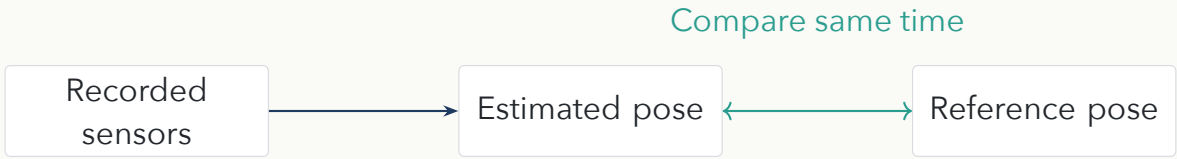
- The camera sends measurements over USB; librealsense exposes them to programs.
- ROS 2 topics carry typed messages, and bags preserve input for repeated tests.
- Explain how the ROS recording route differs from the direct librealsense input program, and what live processing adds.

Try answering aloud before you turn the page.

Ground truth gives an independent answer to compare against

Ground truth is an independently measured reference, with its own uncertainty. A smooth estimated path is not enough: it can still be smoothly wrong.

EuRoC is a public drone dataset with stereo images, IMU samples and reference motion measurements. The machine-hall sequence called MH01 gives us known input before adapting to our camera.



Our table-camera recording needs an independent reference before we can claim absolute accuracy.

Alignment removes an arbitrary origin without hiding scale

Alignment places two trajectories in the same coordinate frame. A **rigid alignment** rotates and shifts the entire estimated path once, without stretching it.



Before: different origins

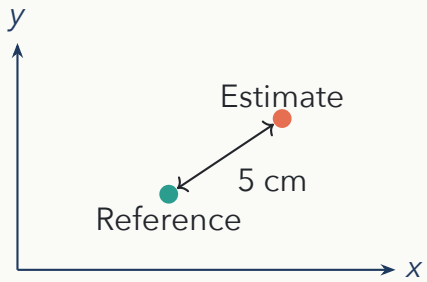
After: one shared frame

Teal shows the reference; coral shows the estimate. SLAM can choose its own world origin, so removing that difference is fair. Stereo already provides distances in metres. Stretching its trajectory would conceal an error in those distances.

Match timestamps and sensor frames first; then use rotation and translation for the main score.

ATE measures the global position mismatch

ATE, absolute trajectory error, compares estimated and reference positions at matched times after alignment. A **residual** is one remaining difference.



Illustrative example: the aligned position differs by 3 cm in one direction and 4 cm in the perpendicular direction.

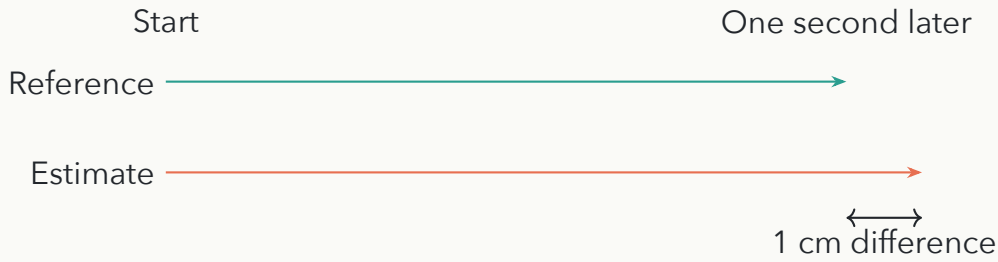
The residual length is $\sqrt{3^2 + 4^2} = 5$ cm.

RMSE, root mean square error, squares every residual length, averages the squares, then takes the square root.

ATE RMSE summarizes the whole matched path; it is not an error bound for every pose.

RPE asks how much a short motion segment is wrong

RPE, relative pose error, compares motion between two times rather than absolute positions. It can measure both displacement error and turning error.



Illustrative drawing, not to scale: reference displacement is 1.00 m, estimate is 1.01 m, with no turning difference. Translational RPE is 1 cm.

Our evaluation selects segments ending at least one second after their start. This describes local drift over that interval, not a constant error rate.

ATE asks about the global path; RPE asks about individual motion segments.

Three EuRoC runs support the build, not our camera

The input is about 184 seconds of drone motion. The evaluated reference span covers about 81 metres of travel. Each run uses the same recorded sensor data.

	Global ATE RMSE	One-second translational RPE RMSE
First run	4.15 cm	1.33 cm
Second run	3.68 cm	1.36 cm
Third run	4.16 cm	1.40 cm

The ORB-SLAM3 paper reports about 3.6 cm ATE on MH01. Our three-run average is about 4.0 cm. The paper reports the median of ten runs: the average of the two middle results when ordered. Evaluation details can differ, so this is only a rough comparison.

The runs show that our Pi executable estimates motion on known input. They do not test D435i calibration or healthy real-time speed; under-voltage affected the Pi.

You can now explain what an accuracy score proves

You should now be able to explain the following in your own words.

- Ground truth must independently measure the quantity we want to test.
- Rigid alignment removes an arbitrary origin; ATE and RPE answer different questions.
- Explain why good EuRoC accuracy does not qualify the D435i, its timing or live speed.

Try answering aloud before you turn the page.

The first gates establish a trustworthy sensor path

A **gate** is a question that must be answered with evidence before depending on that layer. It is like checking power and UART framing before debugging the application protocol.

- **N0 asks whether the Pi has stable power, cooling and a usable platform.** An unstable computer makes later failures ambiguous.
- **N1 asks whether the intended camera has correct USB access.** Its own link must be at least 5000 megabits per second; a slow fallback can limit delivery.
- **N2 asks whether raw samples, directly from the sensor, arrive correctly over time.** A mandatory 300-second continuous run (five minutes) catches sustained faults that a brief preview can miss.
- **N3 asks whether ROS 2 preserves those measurements through recording and replay.** This isolates middleware faults before estimation.

The estimation gates separate algorithm, camera and timing load

- **N4A asks whether the build works on known EuRoC input.** It can be investigated independently of camera transport because the dataset already supplies measurements.
- **N4B asks whether recorded D435i input supports reliable offline estimation.** First check sensor delivery, review adding the second camera stream, and check calibration. Then replay the same recording to investigate estimation.
- **N4C asks whether live estimation keeps up without losing trustworthy input.** It comes after offline review because live scheduling adds another possible failure source.

N2 checks one infrared stream. Stereo needs a second stream and more data delivery, so it needs an additional reviewed test. Passing N2 alone does not test stereo.

Offline replay lets us investigate estimation before adding live timing pressure.

The final gates connect timing evidence to safe use

- **N5 asks where computation time and resources go.** Measure processing, queues and sample age before promising a usable update rate.
- **N6 asks whether unreliable perception is detected and handled.** A pose can be old or invalid. If SLAM restarts its map, the world origin can change; the controller must detect that change before using the pose.
- **N7 asks whether the controller integration behaves safely.** It follows timing and health checks because the STM32H7 must know when a perception input is usable.

Latency is delay between defined events. **Sample age** is how old the measurement is when consumed. A short computation can still use old input if a queue is growing.

Define clock relationships and failure behavior before making a control promise.

Debug from the physical layer upward



If raw samples already have gaps, changing SLAM parameters cannot recover missing measurements. If replay is good but live input falls behind, inspect scheduling and queues.

Our offline D435i program crashed on its first frame. A **pointer** stores an address, as in embedded C. ORB-SLAM3 checked an uninitialized keyframe pointer: it read a garbage value and tried to access that address before any keyframe existed.

The fix sets that pointer to null, meaning no keyframe yet, before the check. The EuRoC example never asked for this value. Power loss interrupted the rebuild, so a successful rerun is still needed to verify the fix.

The next investigation should answer the first unresolved question, not the most exciting one.

You can now explain why the gates have this order

You should now be able to explain the following in your own words.

- N0 through N3 check the computer and sensor delivery; N4 tests known recordings, our camera recordings and live estimation.
- N5 through N7 check delays, detect unusable poses and test safe use by the controller.
- Explain which layer you would inspect if replay succeeds while live tracking falls behind.

Try answering aloud before you turn the page.

Today short camera tests work, but power still limits progress

As of 14 October 2026, the 8 GB Pi runs the same OS card. Both infrared views and IMU samples arrived during a short camera test. The three earlier EuRoC trajectories remain our known-input baseline.

The microSD and tested USB stick are too slow for recording. A short RAM recording worked, with a verified copy afterwards; the longer take still needs testing.

The offline camera program crashed. Its pointer fix is applied, but the rebuild was interrupted. Under-voltage persists on the same supply: more RAM did not fix power. These are diagnostic results, not completed gates.

Next: restore stable power and resume the rebuild, then test a 100-second RAM take and offline replay before a live demo. Calibration, qualified data storage and the required gate checks are still needed before trusting the system.

Camera tests require your explicit “CAMERA AVAILABLE” confirmation.

You can now choose and explain the next useful task

You should now be able to explain the following in your own words.

- Tell the story of one stereo pair and its IMU samples from capture through recording to a pose and a score.
- Explain what the EuRoC tests showed, what our camera still needs to prove, and why controller integration comes later.
- For the next test, explain the question, the measurements needed, and which failure would make you stop.

Try answering aloud before you turn the page.

What to learn before each gate

Gate and its question	Concepts to understand first	One primary source to read	Self-check
N0. Is the Pi stable under load?	Power and cooling; throttling means reduced processor speed.	Raspberry Pi hardware docs: power and cooling.	Why can low voltage make a speed test misleading?
N1. Can we access the intended camera over fast USB?	Device identity; actual USB link speed; permissions.	librealsense troubleshooting guide: USB connection.	Why does a fast USB hub not prove the camera link is fast?
N2. Do raw samples keep arriving correctly?	Sample intervals; gaps; bias; Allan analysis compares noise across averaging times.	Kalibr IMU noise model: Allan analysis.	Why does a short, clean preview not prove sustained delivery?
N3. Does ROS 2 preserve data through recording and replay?	Topics; QoS means delivery rules; tf2 handles frame transforms; bag replay.	ROS 2 tutorials: topics, QoS, tf2 and recording.	What must survive replay besides the image pixels?

What to learn before each gate

Gate and its question	Concepts to understand first	One primary source to read	Self-check
N4. Can we estimate motion on known data, camera replay, then live input?	Features; calibration; IMU preintegration; ground truth; ATE and RPE.	Campos et al. (2021), ORB-SLAM3 paper.	Why does a good EuRoC score not qualify our camera or live speed?
N5. Where do processing time and resources go?	Latency; queue growth; sample age; clock relationships; processor load.	ROS 2 tracing tutorial: measuring execution time.	How can fast processing still deliver an old pose?
N6. Can we detect and handle an unusable pose?	Pose transforms in 3D; map resets; invalid or old measurements.	Barfoot, *State Estimation for Robotics*: pose transforms.	Why must a new map origin be detected before using a pose?
N7. Does controller integration behave safely?	Local deadlines; watchdog timeouts; fallback behavior.	STMicroelectronics STM32H7 independent watchdog training.	If the Pi stops sending poses, which safety loops must stay local?

These terms summarize the ideas you have learned

Pose and trajectory. Position plus orientation, and their history.

Disparity. A point's horizontal shift between stereo views.

Feature and match. A distinctive image region, and its correspondence.

Keyframe and map point. A kept image with its pose, and an estimated scene point.

Bias. A sensor offset that may slowly change.

Preintegration. A summary of IMU motion between image times.

Frame and transform. Axes and origin, and a conversion between them.

Timestamp and clock domain. An event's time and its clock reference.

Intrinsics and extrinsics. Camera projection and relations between sensors.

Alignment. Placing paths in one frame.

ATE and RPE. Global position error and error in a motion segment.

RMSE. The square root of the average squared errors.

Each tool answers a different engineering question

Tool	Purpose
librealsense	Reads camera measurements and SDK recordings.
realsense-ros	Publishes camera data and metadata as ROS 2 messages.
rosbag2	Records and replays ROS 2 topic streams.
ORB-SLAM3	Estimates poses and a map from images and motion samples.
evo	Evaluates estimated trajectories against reference trajectories.
Trajectory evaluator	Calculates the documented local RPE and comparison plots.
Kalibr	Fits sensor geometry and time offset from a measured target.
Allan analysis	Estimates noise and bias behavior from stationary IMU data.

Choose the tool from the question you need to answer.

Use these commands to rebuild the deck or score saved paths

Deck-only reproduction from this repository. These commands do not access the camera.

```
cd slides_learn
latexmk -xelatex -interaction=nonstopmode main.tex
```

```
swift -module-cache-path qa/swift-cache \
  ~/Desktop/OS/OS-project/heterogeneous-edge-ai-os/slides/render.swift \
  main.pdf preview
```

Example command for scoring saved trajectories:

```
evo_ape tum REFERENCE.tum ESTIMATE.tum -a
```

First make times, frames and file formats compatible. The alignment option rotates and shifts; it does not correct sensor timing or establish ground truth. Camera procedures stay in the approved runbook.

The sources separate project evidence from general methods

Project evidence. October 13 session facts and October 14 addendum; three-run evaluation; calibration notes; course, progress and report decks. Page notes hold detailed provenance.

Power and USB. Raspberry Pi hardware docs: power and cooling. RealSense librealsense troubleshooting: USB access.

ROS 2. Official Jazzy tutorials: topics, QoS, tf2 and bags; tracing tutorial: execution time.

SLAM and data. Campos et al. (2021), ORB-SLAM3, including Table II. Burri et al. (2016), EuRoC datasets.

Calibration and noise. Furgale et al. (2013), Unified Temporal and Spatial Calibration. Kalibr camera–IMU guide and IMU noise model, including Allan analysis.

Inertial estimation. Forster et al. (2017), On-Manifold Preintegration for Real-Time Visual-Inertial Odometry.

Trajectory evaluation. Sturm et al. (2012), RGB-D SLAM benchmark: ATE and RPE. Zhang and Scaramuzza (2018), quantitative trajectory evaluation tutorial. Maintainer's evo metric guide.

Frames and safe use. Barfoot, State Estimation for Robotics, second edition: 3D pose transforms, called SE(3). ST STM32H7 watchdog training.

Projector. RealSense, Projectors for D400 Series: texture for stereo depth.

Images. Reused course illustrations are conceptual; new diagrams are schematic. Worked numbers are illustrative.